MODUL PRAKTIKUM BASIS DATA

Pertemuan 6 | Bagian 2 dari 3

Praktikum PostgreSQL: SELECT, JOIN, dan Agregasi

Blok 3 (SELECT Dasar), Blok 4 (JOIN dan Agregasi), dan Blok 5 (UNION, Subquery, EXISTS, ANY/ALL, CASE) dengan database Pasar Nusantara

Keterangan	Isi
Mata kuliah	Basis Data
Isi dokumen ini	**Blok 3** SELECT dasar (30 menit), **Blok 4** JOIN dan agregasi (40 menit), **Blok 5** query lanjutan (20 menit)
Pola belajar	Terpandu: dosen mendemonstrasikan setiap konsep, kalian mengetik dan menjalankan query yang sama, lalu mengerjakan latihan di akhir setiap blok
Database	`pasar_nusantara`: 8 pembeli, 5 toko, 12 produk, 11 pesanan, 21 baris detail pesanan
Persiapan	Buat database `pasar_nusantara`, lalu jalankan skrip `02_pasar_nusantara.sql` (langkah di halaman berikut)
Dokumen sebelumnya / berikutnya	Bagian 1: Pengantar, DDL, DML Bagian 3: Latihan Mandiri dan Tugas

Setelah menyelesaikan bagian ini, kalian mampu:

1. Menulis SELECT dengan kolom pilihan, alias, ekspresi, dan DISTINCT.
2. Menyaring baris dengan WHERE, operator pembanding, AND/OR/NOT, LIKE/ILIKE, IN, BETWEEN, dan IS NULL.
3. Mengurutkan dan membatasi hasil dengan ORDER BY, LIMIT, dan OFFSET.
4. Menggabungkan tabel hasil normalisasi dengan INNER, LEFT, RIGHT, FULL, dan CROSS JOIN.
5. Meringkas data dengan COUNT, SUM, AVG, MIN, MAX, GROUP BY, dan HAVING.
6. Menggunakan UNION, subquery, EXISTS, ANY/ALL, dan CASE untuk pertanyaan yang lebih kompleks.

Persiapan: Memuat Database Pasar Nusantara

Berbeda dengan Blok 2 yang kita ketik baris demi baris, database ini dimuat sekaligus dari file skrip, karena tujuan kita sekarang adalah **membaca** data, bukan membangunnya. Pilih salah satu cara.

Langkah	Lewat pgAdmin 4	Lewat psql
1. Buat database	Klik kanan **Databases** ▸ **Create** ▸ **Database…**, isi `pasar_nusantara`, lalu klik Save	`CREATE DATABASE pasar_nusantara;`
2. Masuk ke database	Klik `pasar_nusantara`, lalu buka **Tools** ▸ **Query Tool**	`\c pasar_nusantara`
3. Jalankan skrip	Klik ikon **Open File** di Query Tool, pilih `02_pasar_nusantara.sql`, lalu tekan **F5**	`\i C:/praktikum/02_pasar_nusantara.sql`
4. Cek hasil	Lihat tab Data Output: harus muncul 8, 5, 12, 11, 21	Sama

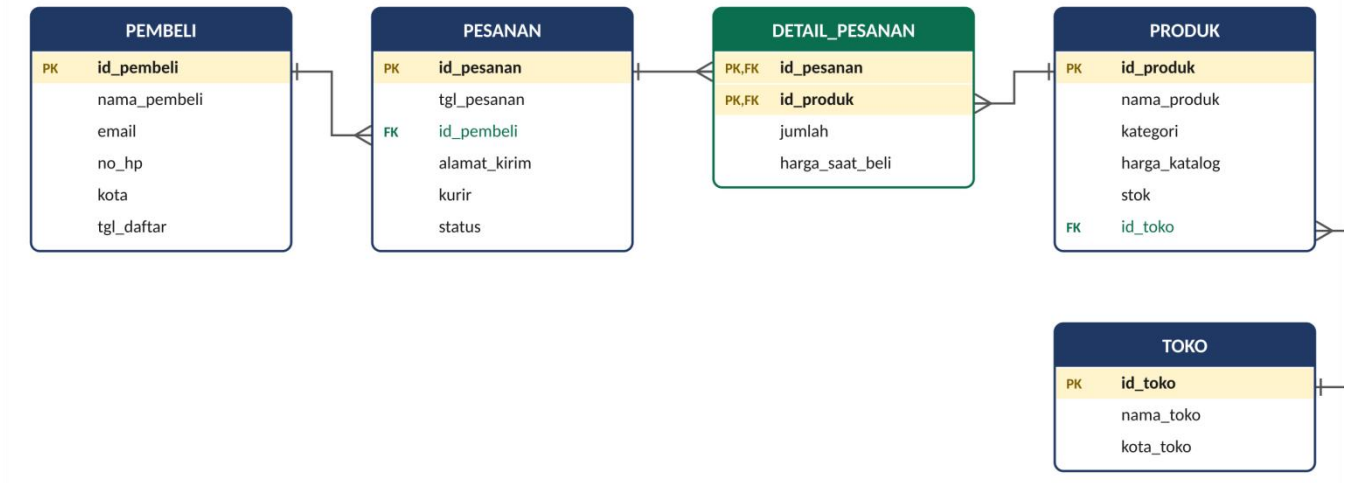
Baris terakhir skrip: cek jumlah data

```
SELECT (SELECT COUNT(*) FROM pembeli) AS pembeli, (SELECT COUNT(*) FROM toko) AS toko,
       (SELECT COUNT(*) FROM produk) AS produk, (SELECT COUNT(*) FROM pesanan) AS pesanan,
       (SELECT COUNT(*) FROM detail_pesanan) AS detail;
```

▸ **Hasil** 1 baris · *jika angkanya berbeda, jalankan ulang skrip*

	pembeli	toko	produk	pesanan	detail
1	8	5	12	11	21

Skemanya adalah hasil normalisasi Contoh C Pertemuan 5, dengan beberapa kolom tambahan supaya latihan query lebih kaya: `no_hp`, `kota`, dan `tgl_daftar` di PEMBELI, `stok` di PRODUK, serta `status` di PESANAN.



Gambar 3.1 Skema database pasar_nusantara

Kenali datanya sebelum bertanya. Empat tabel berikut akan sering kalian lihat kembali. Isi `detail_pesanan` (21 baris) akan kita lihat lewat JOIN di Blok 4.

toko

```
SELECT * FROM toko;
```

▸ **Hasil** 5 baris

	id_ toko	nama_ toko	kota_ toko
1	TK1	Batik Pekalongan Jaya	Pekalongan
2	TK2	Gadget Murah ID	Jakarta
3	TK3	Kopi Gayo Aceh	Takengon
4	TK4	Dapur Bunda	Bandung
5	TK5	Rajut Cantik	Bogor

pembeli

```
SELECT * FROM pembeli;
```

▸ **Hasil** 8 baris

	id_ pembeli	nama_ pembeli	email	no_ hp	kota	tgl_ daftar
1	P01	Nadia Putri	nadia.p@mail.id	0812-1100-2211	Bandung	2025-11-03
2	P02	Fajar Nugroho	fajar.n@mail.id	*[null]*	Semarang	2025-12-15
3	P03	Rizky Ramadhan	rizky.r@mail.id	0813-4455-6677	Jakarta	2026-01-08
4	P04	Maya Sari	maya.s@mail.id	*[null]*	Surabaya	2026-01-20
5	P05	Dimas Pratama	dimas.p@mail.id	0857-3322-1100	Bogor	2026-02-11
6	P06	Anisa Rahmawati	anisa.r@mail.id	0878-9900-1122	Yogyakarta	2026-03-02
7	P07	Yusuf Hakim	yusuf.h@mail.id	*[null]*	Medan	2026-04-18
8	P08	Lestari Wulandari	lestari.w@mail.id	0896-5544-3322	Bogor	2026-05-25

produk

```
SELECT * FROM produk;
```

▸ **Hasil** 12 baris

	id_ produk	nama_ produk	kategori	harga_ katalog	stok	id_ toko
1	PR01	Kemeja Batik Pria	Fashion	185000.00	40	TK1
2	PR02	Kain Batik Tulis	Fashion	350000.00	12	TK1
3	PR03	Earbuds TWS X1	Elektronik	229000.00	25	TK2
4	PR04	Powerbank 10000mAh	Elektronik	159000.00	60	TK2
5	PR05	Kabel USB-C 1m	Elektronik	35000.00	150	TK2
6	PR06	Smartwatch Fit S2	Elektronik	499000.00	8	TK2
7	PR07	Kopi Gayo Arabika 250g	Makanan	95000.00	70	TK3
8	PR08	Kopi Gayo Robusta 250g	Makanan	75000.00	0	TK3
9	PR09	Sambal Roa Botol	Makanan	45000.00	35	TK4
10	PR10	Keripik Tempe Pedas	Makanan	20000.00	100	TK4
11	PR11	Daster Batik Cap	Fashion	120000.00	30	TK1
12	PR12	Mouse Wireless M2	Elektronik	89000.00	45	TK2

pesanan

```
SELECT * FROM pesanan;
```

▸ **Hasil** 11 baris

	id_ pesanan	tgl_ pesanan	id_ pembeli	alamat_ kirim	kurir	status
1	ORD-001	2026-05-02	P01	Jl. Dago 21, Bandung	JNE	Selesai
2	ORD-002	2026-05-03	P02	Jl. Pemuda 8, Semarang	SiCepat	Selesai
3	ORD-003	2026-05-20	P01	Jl. A. Yani 3, Cirebon	J&T	Selesai
4	ORD-004	2026-05-22	P03	Jl. Kemang Raya 10, Jakarta	JNE	Selesai
5	ORD-005	2026-06-01	P04	Jl. Darmo 45, Surabaya	SiCepat	Selesai
6	ORD-006	2026-06-05	P03	Jl. Kemang Raya 10, Jakarta	J&T	Dibatalkan
7	ORD-007	2026-06-10	P05	Jl. Pajajaran 7, Bogor	JNE	Selesai
8	ORD-008	2026-06-14	P06	Jl. Kaliurang 12, Yogyakarta	AnterAja	Dikirim
9	ORD-009	2026-06-18	P02	Jl. Pemuda 8, Semarang	SiCepat	Dikirim
10	ORD-010	2026-06-20	P04	Jl. Darmo 45, Surabaya	JNE	Diproses
11	ORD-011	2026-06-21	P01	Jl. Dago 21, Bandung	AnterAja	Diproses

BLOK 3 | 30 MENIT | TERPANDU

SELECT Dasar: Bertanya pada Satu Tabel

Memilih kolom, menyaring baris, mencocokkan pola, menangani NULL, mengurutkan, dan membatasi hasil

3.1 SELECT, Alias, dan Ekspresi

Bentuk dasar SELECT adalah `SELECT kolom-kolom FROM tabel`. Sebutkan kolom yang dibutuhkan saja, dipisahkan koma. `SELECT *` praktis untuk mengintip data, tetapi di aplikasi sungguhan sebaiknya kolom disebut satu per satu supaya hasilnya jelas dan hemat.

Memilih kolom

```
SELECT nama_produk, kategori, harga_katalog
FROM produk;
```

▸ **Hasil** 12 baris (ditampilkan 5 baris pertama)

	nama_ produk	kategori	harga_ katalog
1	Kemeja Batik Pria	Fashion	185000.00
2	Kain Batik Tulis	Fashion	350000.00
3	Earbuds TWS X1	Elektronik	229000.00
4	Powerbank 10000mAh	Elektronik	159000.00
5	Kabel USB-C 1m	Elektronik	35000.00

Kolom hasil juga bisa berupa **perhitungan**, dan kolom apa pun bisa diberi nama baru (**alias**) dengan `AS`. Di bawah ini kita menghitung nilai stok (harga × stok) serta harga setelah diskon 10%. Operator `||` menggabungkan teks.

Ekspresi dan alias

```
SELECT nama_produk,
       harga_katalog * stok        AS nilai_stok,
       harga_katalog * 0.9         AS harga_diskon,
       nama_produk || ' - ' || kategori AS label
FROM produk;
```

▸ **Hasil** 12 baris (ditampilkan 5 baris pertama)

	nama_ produk	nilai_ stok	harga_ diskon	label
1	Kemeja Batik Pria	7400000.00	166500.000	Kemeja Batik Pria - Fashion
2	Kain Batik Tulis	4200000.00	315000.000	Kain Batik Tulis - Fashion
3	Earbuds TWS X1	5725000.00	206100.000	Earbuds TWS X1 - Elektronik
4	Powerbank 10000mAh	9540000.00	143100.000	Powerbank 10000mAh - Elektronik
5	Kabel USB-C 1m	5250000.00	31500.000	Kabel USB-C 1m - Elektronik

TIPS | Alias dengan spasi atau huruf besar

Alias ditulis tanpa petik jika hanya berisi huruf kecil, angka, dan garis bawah (`nilai_stok`). Jika ingin spasi atau huruf besar, gunakan **petik ganda**: `AS "Nilai Stok"`. Petik tunggal di posisi alias akan menghasilkan error.

3.2 DISTINCT: Menghilangkan Duplikat

Ada 12 produk, tetapi berapa kategorinya? `DISTINCT` membuang baris hasil yang kembar.

Daftar kategori unik

```
SELECT DISTINCT kategori
FROM produk
ORDER BY kategori;
```

▸ **Hasil** 3 baris

	kategori
1	Elektronik
2	Fashion
3	Makanan

Tanpa `ORDER BY`, urutan baris hasil **tidak dijamin**. PostgreSQL boleh mengembalikannya dalam urutan apa pun, dan urutannya bisa berubah setelah data diubah. Jika urutan penting, selalu tulis ORDER BY.

3.3 WHERE dan Operator Pembanding

`WHERE` menyaring baris. Hanya baris yang kondisinya bernilai **benar** yang ikut ke hasil. Operator pembanding yang tersedia: `=` (sama dengan), `<>` atau `!=` (tidak sama dengan), `<`, `>`, `<=`, dan `>=`.

Produk yang stoknya habis

```
SELECT nama_produk, stok
FROM produk
WHERE stok = 0;
```

▸ **Hasil** 1 baris

	nama_ produk	stok
1	Kopi Gayo Robusta 250g	0

Dua syarat dengan AND

```
SELECT nama_produk, harga_katalog, stok
FROM produk
WHERE kategori = 'Elektronik' AND harga_katalog < 200000;
```

▸ **Hasil** 3 baris

	nama_ produk	harga_ katalog	stok
1	Powerbank 10000mAh	159000.00	60
2	Kabel USB-C 1m	35000.00	150
3	Mouse Wireless M2	89000.00	45

3.4 AND, OR, NOT, dan Jebakan Urutan

`AND` berarti kedua syarat harus benar. `OR` berarti cukup salah satu syarat yang benar. `NOT` membalik syarat. Hati-hati: **AND dikerjakan lebih dulu daripada OR**, sama seperti perkalian dikerjakan sebelum penjumlahan. Misalkan kita ingin "produk Makanan atau Fashion yang harganya di atas 150 ribu":

Versi SALAH

```
SELECT nama_produk, kategori, harga_katalog
FROM produk
WHERE kategori = 'Makanan' OR kategori = 'Fashion' AND harga_katalog > 150000;
```

▸ **Hasil** 6 baris · *Kopi dan Sambal seharga puluhan ribu ikut muncul*

	nama_ produk	kategori	harga_ katalog
1	Kemeja Batik Pria	Fashion	185000.00
2	Kain Batik Tulis	Fashion	350000.00
3	Kopi Gayo Arabika 250g	Makanan	95000.00
4	Kopi Gayo Robusta 250g	Makanan	75000.00
5	Sambal Roa Botol	Makanan	45000.00
6	Keripik Tempe Pedas	Makanan	20000.00

PostgreSQL membacanya sebagai `Makanan OR (Fashion AND > 150000)`, sehingga semua Makanan ikut muncul berapa pun harganya. Tanda kurung memperbaiki maksudnya:

Versi BENAR

```
SELECT nama_produk, kategori, harga_katalog
FROM produk
WHERE (kategori = 'Makanan' OR kategori = 'Fashion') AND harga_katalog > 150000;
```

▸ **Hasil** 2 baris

	nama_ produk	kategori	harga_ katalog
1	Kemeja Batik Pria	Fashion	185000.00
2	Kain Batik Tulis	Fashion	350000.00

KUNCI KONSEP | Aturan praktis

Setiap kali AND dan OR muncul bersama dalam satu WHERE, **selalu pakai tanda kurung**, meskipun kalian yakin urutannya sudah benar. Kurung membuat maksud query jelas bagi PostgreSQL dan bagi teman yang membaca kode kalian.

3.5 LIKE dan ILIKE: Mencocokkan Pola Teks

Untuk pencarian teks yang tidak persis sama, gunakan `LIKE` dengan dua karakter khusus: `%` berarti "nol atau lebih karakter apa saja", dan `_` berarti "tepat satu karakter apa saja".

Pola	Cocok dengan	Contoh yang cocok
`'Kopi%'`	diawali "Kopi"	Kopi Gayo Arabika 250g
`'%250g'`	diakhiri "250g"	Kopi Gayo Robusta 250g
`'%Batik%'`	mengandung "Batik" di mana saja	Kemeja Batik Pria, Daster Batik Cap
`'PR1_'`	"PR1" diikuti tepat satu karakter	PR10, PR11, PR12 (bukan PR1)

Garis bawah: tepat satu karakter

```
SELECT id_produk, nama_produk
FROM produk
WHERE id_produk LIKE 'PR1_';
```

▸ **Hasil** 3 baris

	id_ produk	nama_ produk
1	PR10	Keripik Tempe Pedas
2	PR11	Daster Batik Cap
3	PR12	Mouse Wireless M2

Sekarang cari produk yang mengandung kata "batik" dengan huruf kecil:

LIKE peka huruf besar-kecil

```
SELECT nama_produk FROM produk WHERE nama_produk LIKE '%batik%';
```

▸ **Hasil** 0 baris · *tidak ada satu pun, karena di data tertulis "Batik"*

	nama_ produk

ILIKE mengabaikan huruf besar-kecil

```
SELECT nama_produk FROM produk WHERE nama_produk ILIKE '%batik%';
```

▸ **Hasil** 3 baris

	nama_ produk
1	Kemeja Batik Pria
2	Kain Batik Tulis
3	Daster Batik Cap

Di PostgreSQL, `LIKE` **peka** huruf besar-kecil, sedangkan `ILIKE` (*insensitive LIKE*) tidak. Untuk fitur pencarian yang diketik pengguna, `ILIKE` hampir selalu yang kalian butuhkan.

3.6 IN dan BETWEEN

`IN (daftar)` adalah cara ringkas menulis banyak `OR` untuk kolom yang sama. `BETWEEN a AND b` memilih nilai dari a sampai b, **termasuk kedua ujungnya**. BETWEEN berlaku untuk angka maupun tanggal.

IN dan NOT IN

```
SELECT id_pesanan, kurir, status
FROM pesanan
WHERE kurir IN ('JNE', 'J&T')
  AND status NOT IN ('Dibatalkan');
```

▸ **Hasil** 5 baris

	id_ pesanan	kurir	status
1	ORD-001	JNE	Selesai
2	ORD-003	J&T	Selesai
3	ORD-004	JNE	Selesai
4	ORD-007	JNE	Selesai
5	ORD-010	JNE	Diproses

BETWEEN pada tanggal

```
SELECT id_pesanan, tgl_pesanan, status
FROM pesanan
WHERE tgl_pesanan BETWEEN '2026-06-01' AND '2026-06-15';
```

▸ **Hasil** 4 baris · *1 Juni dan 15 Juni ikut dihitung*

	id_ pesanan	tgl_ pesanan	status
1	ORD-005	2026-06-01	Selesai
2	ORD-006	2026-06-05	Dibatalkan
3	ORD-007	2026-06-10	Selesai
4	ORD-008	2026-06-14	Dikirim

3.7 NULL: Nilai yang Belum Diketahui

`NULL` bukan nol dan bukan teks kosong. NULL berarti **tidak diketahui**. Beberapa pembeli belum mengisi nomor HP. Coba cari mereka dengan cara yang terasa wajar:

Cara yang SALAH

```
SELECT nama_pembeli, no_hp FROM pembeli WHERE no_hp = NULL;
```

▸ **Hasil** 0 baris · *0 baris, padahal ada 3 pembeli tanpa HP*

	nama_ pembeli	no_ hp

Hasilnya kosong. Membandingkan apa pun dengan NULL memakai `=` menghasilkan "tidak diketahui", bukan "benar", sehingga tidak ada baris yang lolos. Analoginya: "Apakah umur orang yang belum kita

kenal sama dengan 20?" Jawabannya bukan ya, bukan juga tidak, melainkan *tidak tahu*. Untuk NULL gunakan operator khusus `IS NULL` dan `IS NOT NULL`:

Cara yang BENAR

```
SELECT nama_pembeli, kota, no_hp
FROM pembeli
WHERE no_hp IS NULL;
```

▸ **Hasil** 3 baris

	nama_ pembeli	kota	no_ hp
1	Fajar Nugroho	Semarang	*[null]*
2	Maya Sari	Surabaya	*[null]*
3	Yusuf Hakim	Medan	*[null]*

Untuk tampilan laporan, fungsi `COALESCE(kolom, pengganti)` mengganti NULL dengan nilai lain, misalnya `COALESCE(no_hp, '-')` agar kolom kosong tampil sebagai tanda strip.

3.8 ORDER BY, LIMIT, dan OFFSET

`ORDER BY` mengurutkan hasil: `ASC` untuk naik (bawaan) dan `DESC` untuk turun. Bisa lebih dari satu kolom. Kolom kedua dipakai sebagai penentu jika nilai kolom pertama sama. `LIMIT n` membatasi jumlah baris, dan `OFFSET m` melewati m baris pertama.

Urut kategori, lalu harga termahal di tiap kategori

```
SELECT nama_produk, kategori, harga_katalog
FROM produk
ORDER BY kategori ASC, harga_katalog DESC;
```

▸ **Hasil** 12 baris (ditampilkan 6 baris pertama)

	nama_ produk	kategori	harga_ katalog
1	Smartwatch Fit S2	Elektronik	499000.00
2	Earbuds TWS X1	Elektronik	229000.00
3	Powerbank 10000mAh	Elektronik	159000.00
4	Mouse Wireless M2	Elektronik	89000.00
5	Kabel USB-C 1m	Elektronik	35000.00
6	Kain Batik Tulis	Fashion	350000.00

Tiga produk termahal

```
SELECT nama_produk, harga_katalog
FROM produk
ORDER BY harga_katalog DESC
LIMIT 3;
```

▸ **Hasil** 3 baris

	nama_ produk	harga_ katalog
1	Smartwatch Fit S2	499000.00

	nama_ produk	harga_ katalog
2	Kain Batik Tulis	350000.00
3	Earbuds TWS X1	229000.00

Halaman ke-2 (4 produk per halaman)

```
SELECT id_produk, nama_produk
FROM produk
ORDER BY id_produk
LIMIT 4 OFFSET 4;
```

▸ **Hasil** 4 baris

	id_ produk	nama_ produk
1	PR05	Kabel USB-C 1m
2	PR06	Smartwatch Fit S2
3	PR07	Kopi Gayo Arabika 250g
4	PR08	Kopi Gayo Robusta 250g

INFO | LIMIT/OFFSET = pagination

Halaman katalog marketplace yang menampilkan "20 produk per halaman" bekerja persis seperti ini: halaman ke-n memakai `LIMIT 20 OFFSET (n-1)*20`. Di Laravel, `Produk::orderBy('id_produk')->paginate(20)` menghasilkan SQL dengan LIMIT dan OFFSET yang sama. Tanpa ORDER BY, isi setiap halaman bisa berubah-ubah.

3.9 Latihan Blok 3 (8 Menit)

LATIHAN | Tulis query untuk setiap pertanyaan

1. Nama dan email semua pembeli yang tinggal di **Bogor**.
2. Produk kategori **Makanan** yang stoknya **kurang dari 50**, urutkan dari stok paling sedikit.
3. Pesanan bulan **Mei 2026** yang dikirim dengan **JNE atau J&T**. Tampilkan id, tanggal, dan kurir.
4. Tiga pembeli pertama (urut nama A–Z) yang nomor HP-nya **sudah terisi**.
5. Produk yang namanya mengandung kata **"kopi"** (huruf besar atau kecil), beserta nilai stoknya (harga × stok) dengan alias `nilai_stok`.

PEMBAHASAN

Soal 1

```
SELECT nama_pembeli, email FROM pembeli WHERE kota = 'Bogor';
```

▸ **Hasil** 2 baris

	nama_ pembeli	email
1	Dimas Pratama	dimas.p@mail.id
2	Lestari Wulandari	lestari.w@mail.id

Soal 2

```
SELECT nama_produk, stok FROM produk
WHERE kategori = 'Makanan' AND stok < 50
ORDER BY stok;
```

▸ **Hasil** 2 baris

	nama_ produk	stok
1	Kopi Gayo Robusta 250g	0
2	Sambal Roa Botol	35

Soal 3

```
SELECT id_pesanan, tgl_pesanan, kurir FROM pesanan
WHERE tgl_pesanan BETWEEN '2026-05-01' AND '2026-05-31'
  AND kurir IN ('JNE', 'J&T');
```

▸ **Hasil** 3 baris

	id_ pesanan	tgl_ pesanan	kurir
1	ORD-001	2026-05-02	JNE
2	ORD-003	2026-05-20	J&T
3	ORD-004	2026-05-22	JNE

Soal 4

```
SELECT nama_pembeli, no_hp FROM pembeli
WHERE no_hp IS NOT NULL
ORDER BY nama_pembeli
LIMIT 3;
```

▸ **Hasil** 3 baris

	nama_ pembeli	no_ hp
1	Anisa Rahmawati	0878-9900-1122
2	Dimas Pratama	0857-3322-1100
3	Lestari Wulandari	0896-5544-3322

Soal 5

```
SELECT nama_produk, harga_katalog * stok AS nilai_stok
FROM produk
WHERE nama_produk ILIKE '%kopi%';
```

▸ **Hasil** 2 baris · *Robusta bernilai 0 karena stoknya habis*

	nama_ produk	nilai_ stok
1	Kopi Gayo Arabika 250g	6650000.00
2	Kopi Gayo Robusta 250g	0.00

BLOK 4 | 40 MENIT | TERPANDU

JOIN dan Agregasi: Menyatukan Kembali dan Meringkas

INNER, LEFT, RIGHT, FULL, CROSS JOIN; COUNT, SUM, AVG, MIN, MAX; GROUP BY dan HAVING

4.1 INNER JOIN: Menyambung Tabel lewat FK

Tabel `produk` hanya menyimpan `id_toko`, bukan nama toko. Untuk menampilkan nama toko, kita sambungkan `produk` dengan `toko` pada kolom yang sama: `produk.id_toko = toko.id_toko`. Inilah pasangan FK–PK yang kalian gambar di skema.

Produk beserta tokonya

```
SELECT p.nama_produk, p.harga_katalog, t.nama_toko, t.kota_toko
FROM produk p
INNER JOIN toko t ON t.id_toko = p.id_toko
ORDER BY t.nama_toko, p.nama_produk;
```

▸ **Hasil** 12 baris

	nama_ produk	harga_ katalog	nama_ toko	kota_ toko
1	Daster Batik Cap	120000.00	Batik Pekalongan Jaya	Pekalongan
2	Kain Batik Tulis	350000.00	Batik Pekalongan Jaya	Pekalongan
3	Kemeja Batik Pria	185000.00	Batik Pekalongan Jaya	Pekalongan
4	Keripik Tempe Pedas	20000.00	Dapur Bunda	Bandung
5	Sambal Roa Botol	45000.00	Dapur Bunda	Bandung
6	Earbuds TWS X1	229000.00	Gadget Murah ID	Jakarta
7	Kabel USB-C 1m	35000.00	Gadget Murah ID	Jakarta
8	Mouse Wireless M2	89000.00	Gadget Murah ID	Jakarta
9	Powerbank 10000mAh	159000.00	Gadget Murah ID	Jakarta
10	Smartwatch Fit S2	499000.00	Gadget Murah ID	Jakarta
11	Kopi Gayo Arabika 250g	95000.00	Kopi Gayo Aceh	Takengon
12	Kopi Gayo Robusta 250g	75000.00	Kopi Gayo Aceh	Takengon

Tiga hal yang perlu diperhatikan:

1. **Alias tabel** `p` dan `t` menyingkat penulisan. `p.nama_produk` artinya kolom `nama_produk` milik tabel `p`.
2. `ON` menyebut syarat penyambungan. Hampir selalu berbentuk *FK = PK*.
3. Kata `INNER` boleh dihilangkan. `JOIN` saja sudah berarti INNER JOIN.

Jika nama kolom ada di dua tabel dan tidak diberi awalan

```
SELECT nama_produk, nama_toko, id_toko
FROM produk p JOIN toko t ON t.id_toko = p.id_toko;
```

▸ **Pesan error dari PostgreSQL**

```
ERROR:  column reference "id_toko" is ambiguous
LINE 1: SELECT nama_produk, nama_toko, id_toko
                            ^
```

Kolom `id_toko` ada di kedua tabel, sehingga PostgreSQL tidak tahu mana yang dimaksud (*ambiguous*). Solusinya: tulis `p.id_toko` atau `t.id_toko`.

4.2 JOIN Banyak Tabel: Menyusun Nota

Untuk rincian pesanan lengkap, kita butuh empat tabel: `pesanan` → `pembeli` (siapa), `detail_pesanan` (apa dan berapa), serta `produk` (namanya). Tambahkan JOIN satu per satu, masing-masing dengan ON-nya sendiri. Perhatikan kolom `subtotal`: di Pertemuan 5 kolom ini **dibuang dari tabel** karena merupakan atribut turunan. Sekarang ia **dihitung** saat dibutuhkan.

Rincian dua pesanan Nadia

```
SELECT ps.id_pesanan, ps.tgl_pesanan, pb.nama_pembeli, pr.nama_produk,
       d.jumlah, d.harga_saat_beli,
       d.jumlah * d.harga_saat_beli AS subtotal
FROM pesanan ps
JOIN pembeli pb        ON pb.id_pembeli = ps.id_pembeli
JOIN detail_pesanan d  ON d.id_pesanan  = ps.id_pesanan
JOIN produk pr         ON pr.id_produk  = d.id_produk
WHERE ps.id_pesanan IN ('ORD-001', 'ORD-003')
ORDER BY ps.id_pesanan, pr.nama_produk;
```

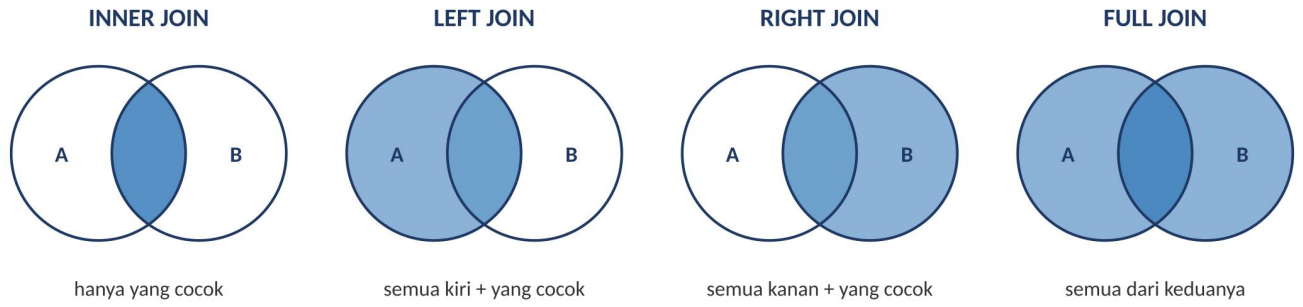
▸ **Hasil** 5 baris

	id_ pesanan	tgl_ pesanan	nama_ pembeli	nama_ produk	jumlah	harga_ saat_ beli	subtotal
1	ORD-001	2026-05-02	Nadia Putri	Earbuds TWS X1	1	229000.00	229000.00
2	ORD-001	2026-05-02	Nadia Putri	Kemeja Batik Pria	2	175000.00	350000.00
3	ORD-003	2026-05-20	Nadia Putri	Kain Batik Tulis	1	350000.00	350000.00
4	ORD-003	2026-05-20	Nadia Putri	Kemeja Batik Pria	1	185000.00	185000.00
5	ORD-003	2026-05-20	Nadia Putri	Powerbank 10000mAh	2	149000.00	298000.00

Hasil ini sama persis dengan data mentah UNF Pasar Nusantara di Pertemuan 5. Perhatikan Kemeja Batik Pria: dibeli Rp175.000 di ORD-001 dan Rp185.000 di ORD-003. Karena `harga_saat_beli` disimpan per transaksi, riwayat harga tetap benar meskipun harga katalog sudah berubah.

4.3 LEFT, RIGHT, dan FULL JOIN

INNER JOIN hanya menampilkan baris yang **punya pasangan** di kedua tabel. Bagaimana dengan pembeli yang belum pernah memesan? Mereka hilang dari hasil INNER JOIN. Untuk pertanyaan seperti "siapa yang belum…?" kita butuh **OUTER JOIN**.



Gambar 4.1 Empat jenis JOIN. Area biru menunjukkan baris yang ikut ke hasil.

LEFT JOIN: semua pembeli, ada atau tidak ada pesanannya

```
SELECT pb.id_pembeli, pb.nama_pembeli, ps.id_pesanan
FROM pembeli pb
LEFT JOIN pesanan ps ON ps.id_pembeli = pb.id_pembeli
ORDER BY pb.id_pembeli, ps.id_pesanan;
```

▸ **Hasil** 13 baris

	id_ pembeli	nama_ pembeli	id_ pesanan
1	P01	Nadia Putri	ORD-001
2	P01	Nadia Putri	ORD-003
3	P01	Nadia Putri	ORD-011
4	P02	Fajar Nugroho	ORD-002
5	P02	Fajar Nugroho	ORD-009
6	P03	Rizky Ramadhan	ORD-004
7	P03	Rizky Ramadhan	ORD-006
8	P04	Maya Sari	ORD-005
9	P04	Maya Sari	ORD-010
10	P05	Dimas Pratama	ORD-007
11	P06	Anisa Rahmawati	ORD-008
12	P07	Yusuf Hakim	*[null]*
13	P08	Lestari Wulandari	*[null]*

P07 Yusuf dan P08 Lestari tetap muncul, dengan `id_pesanan` bernilai NULL karena tidak ada pasangannya. Dengan menyaring NULL tersebut, kita mendapat jawaban untuk "pembeli yang **belum pernah** memesan". Pola ini sangat sering dipakai dan dikenal sebagai **anti-join**.

Pembeli yang belum pernah memesan

```
SELECT pb.id_pembeli, pb.nama_pembeli, pb.tgl_daftar
FROM pembeli pb
LEFT JOIN pesanan ps ON ps.id_pembeli = pb.id_pembeli
WHERE ps.id_pesanan IS NULL;
```

▸ **Hasil** 2 baris · *target promo "pesanan pertama"*

	id_ pembeli	nama_ pembeli	tgl_ daftar
1	P07	Yusuf Hakim	2026-04-18
2	P08	Lestari Wulandari	2026-05-25

Produk yang belum pernah terjual

```
SELECT pr.id_produk, pr.nama_produk, pr.stok
FROM produk pr
LEFT JOIN detail_pesanan d ON d.id_produk = pr.id_produk
WHERE d.id_produk IS NULL;
```

▸ **Hasil** 1 baris

	id_ produk	nama_ produk	stok
1	PR12	Mouse Wireless M2	45

RIGHT JOIN adalah cerminan LEFT JOIN: semua baris tabel **kanan** dipertahankan. `A RIGHT JOIN B` sama dengan `B LEFT JOIN A`, jadi dalam praktik kebanyakan orang cukup memakai LEFT JOIN dan menukar urutan tabel. **FULL JOIN** mempertahankan semua baris dari kedua sisi. Dengan FULL JOIN kita bisa melihat toko yang belum punya produk sekaligus produk yang tidak punya toko (yang terakhir ini mustahil di sini karena dijaga FK NOT NULL).

RIGHT JOIN: toko yang belum punya produk

```
SELECT t.nama_toko, p.nama_produk
FROM produk p
RIGHT JOIN toko t ON t.id_toko = p.id_toko
WHERE p.id_produk IS NULL;
```

▸ **Hasil** 1 baris

	nama_ toko	nama_ produk
1	Rajut Cantik	*[null]*

FULL JOIN digabung dengan COUNT (dibahas di 4.5)

```
SELECT t.id_toko, t.nama_toko, COUNT(p.id_produk) AS jml_produk
FROM toko t
FULL JOIN produk p ON p.id_toko = t.id_toko
GROUP BY t.id_toko, t.nama_toko
ORDER BY t.id_toko;
```

▸ **Hasil** 5 baris

	id_ toko	nama_ toko	jml_ produk
1	TK1	Batik Pekalongan Jaya	3
2	TK2	Gadget Murah ID	5
3	TK3	Kopi Gayo Aceh	2
4	TK4	Dapur Bunda	2
5	TK5	Rajut Cantik	0

CROSS JOIN

CROSS JOIN memasangkan **setiap** baris kiri dengan **setiap** baris kanan tanpa syarat. Hasilnya adalah perkalian jumlah baris. Gunanya terbatas, misalnya untuk membuat kerangka laporan semua kombinasi. Contoh: 3 kategori × 4 kurir = 12 kombinasi.

Semua kombinasi kategori × kurir

```
SELECT k.kategori, u.kurir
FROM (SELECT DISTINCT kategori FROM produk) k
CROSS JOIN (SELECT DISTINCT kurir FROM pesanan) u
ORDER BY k.kategori, u.kurir;
```

▸ **Hasil** 12 baris (ditampilkan 5 baris pertama)

	kategori	kurir
1	Elektronik	AnterAja
2	Elektronik	J&T
3	Elektronik	JNE
4	Elektronik	SiCepat
5	Fashion	AnterAja

PERHATIAN | CROSS JOIN yang tidak disengaja

Jika syarat `ON` terlupa, atau tabel ditulis `FROM a, b` tanpa WHERE penghubung, hasilnya menjadi CROSS JOIN. 11 pesanan × 21 detail menghasilkan 231 baris sampah yang kelihatannya meyakinkan. Jika jumlah baris hasil JOIN jauh lebih banyak dari dugaan, periksa ON kalian.

4.4 Fungsi Agregat

Fungsi agregat meringkas banyak baris menjadi **satu nilai**: `COUNT` (menghitung), `SUM` (menjumlah), `AVG` (rata-rata), `MIN` (terkecil), dan `MAX` (terbesar).

Ringkasan seluruh produk

```
SELECT COUNT(*)                     AS jml_produk,
       MIN(harga_katalog)           AS termurah,
       MAX(harga_katalog)           AS termahal,
       ROUND(AVG(harga_katalog), 2) AS rata_rata,
       SUM(stok)                    AS total_stok
FROM produk;
```

▸ **Hasil** 1 baris

	jml_ produk	termurah	termahal	rata_ rata	total_ stok
1	12	20000.00	499000.00	158416.67	575

COUNT(*) vs COUNT(kolom)

```
SELECT COUNT(*) AS jumlah_pembeli, COUNT(no_hp) AS punya_no_hp
FROM pembeli;
```

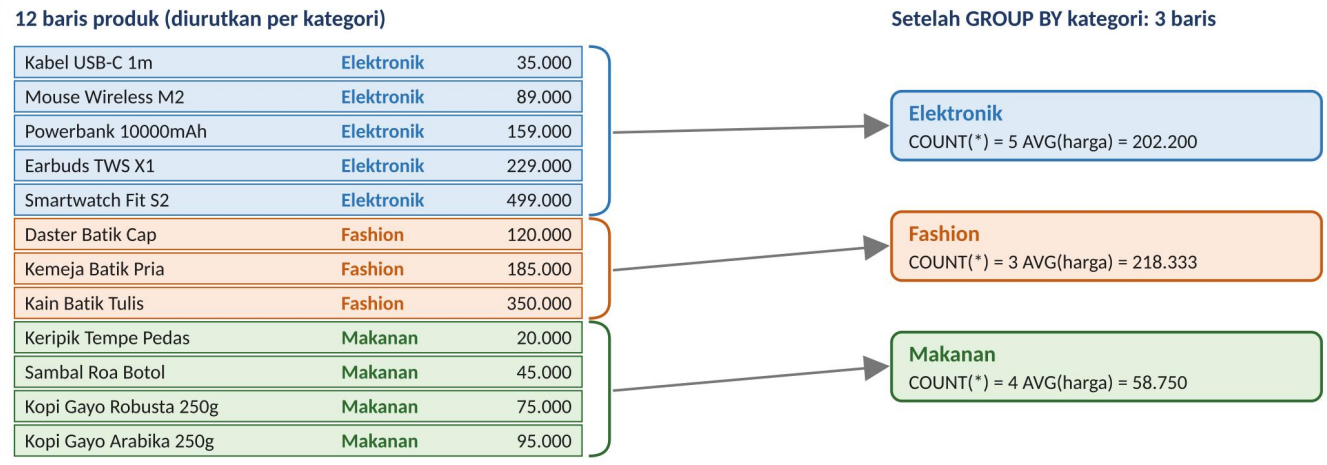
▸ **Hasil** 1 baris

	jumlah_ pembeli	punya_ no_ hp
1	8	5

`COUNT(*)` menghitung **baris**, sedangkan `COUNT(kolom)` menghitung nilai yang **bukan NULL** di kolom itu. Perbedaan ini penting dan akan muncul lagi di latihan. Fungsi agregat lain (SUM, AVG, MIN, MAX) juga mengabaikan NULL.

4.5 GROUP BY: Ringkasan per Kelompok

Bagaimana jika kita ingin rata-rata harga **per kategori**, bukan untuk semua produk? `GROUP BY` memilah baris ke dalam kelompok-kelompok berdasarkan nilai kolom tertentu, lalu fungsi agregat dihitung untuk **setiap kelompok**. Bayangkan menyortir struk belanja ke dalam beberapa kotak sesuai kategori, lalu menghitung isi setiap kotak.



Gambar 4.2 GROUP BY kategori memilah 12 baris menjadi 3 kelompok, lalu setiap kelompok diringkas menjadi satu baris

Jumlah dan rata-rata harga per kategori

```
SELECT kategori,
       COUNT(*)                    AS jml_produk,
       ROUND(AVG(harga_katalog))   AS rata_harga
FROM produk
GROUP BY kategori
ORDER BY kategori;
```

▸ **Hasil** 3 baris

	kategori	jml_ produk	rata_ harga
1	Elektronik	5	202200
2	Fashion	3	218333
3	Makanan	4	58750

Contoh yang paling penting: menghitung **total per pesanan**. Di Pertemuan 5 kolom `total_pesanan` kita buang karena bisa dihitung. Beginilah cara menghitungnya:

Total setiap pesanan

```
SELECT id_pesanan,
       COUNT(*)                      AS jml_item,
       SUM(jumlah * harga_saat_beli) AS total
FROM detail_pesanan
GROUP BY id_pesanan
ORDER BY id_pesanan;
```

▸ **Hasil** 11 baris · *ORD-003 = Rp833.000, sama dengan Pertemuan 5*

	id_pesanan	jml_item	total
1	ORD-001	2	579000.00
2	ORD-002	1	159000.00
3	ORD-003	3	833000.00
4	ORD-004	2	569000.00
5	ORD-005	3	475000.00
6	ORD-006	1	229000.00
7	ORD-007	2	315000.00
8	ORD-008	2	240000.00
9	ORD-009	2	324000.00
10	ORD-010	1	350000.00
11	ORD-011	2	275000.00

KUNCI KONSEP | Aturan emas GROUP BY

Setiap kolom di SELECT harus memenuhi salah satu syarat: **(a) disebut di GROUP BY**, atau **(b) dibungkus fungsi agregat**. Alasannya logis. Satu kelompok hanya menjadi satu baris hasil, sehingga kolom lain yang nilainya berbeda-beda di dalam kelompok itu tidak bisa ditampilkan. Nama produk mana yang harus ditampilkan untuk kelompok "Fashion"? Karena itu PostgreSQL menolak:

```
SELECT kategori, nama_produk, COUNT(*)
FROM produk
GROUP BY kategori;
```

▸ **Pesan error dari PostgreSQL**

```
ERROR:  column "produk.nama_produk" must appear in the GROUP BY clause or be used in an aggregate function
LINE 1: SELECT kategori, nama_produk, COUNT(*)
                         ^
```

GROUP BY bersama JOIN dan WHERE

Pertanyaan bisnis yang nyata biasanya menggabungkan semuanya. Contohnya: **omzet setiap toko**, tanpa menghitung pesanan yang dibatalkan. Urutan berpikirnya: sambungkan tabel yang dibutuhkan (JOIN), buang baris yang tidak relevan (WHERE), kelompokkan (GROUP BY), hitung (SUM), lalu urutkan.

Omzet per toko

```
SELECT t.nama_toko,
       COUNT(DISTINCT d.id_pesanan)       AS jml_pesanan,
       SUM(d.jumlah * d.harga_saat_beli)  AS omzet
FROM toko t
JOIN produk p          ON p.id_toko     = t.id_toko
JOIN detail_pesanan d  ON d.id_produk   = p.id_produk
JOIN pesanan ps        ON ps.id_pesanan = d.id_pesanan
WHERE ps.status <> 'Dibatalkan'
GROUP BY t.nama_toko
ORDER BY omzet DESC;
```

▸ **Hasil** 4 baris · *Rajut Cantik tidak muncul karena INNER JOIN*

	nama_ toko	jml_ pesanan	omzet
1	Gadget Murah ID	5	1579000.00
2	Batik Pekalongan Jaya	4	1475000.00
3	Kopi Gayo Aceh	4	635000.00
4	Dapur Bunda	3	430000.00

`COUNT(DISTINCT kolom)` menghitung nilai yang **berbeda** saja. Tanpa DISTINCT, satu pesanan yang membeli dua produk dari toko yang sama akan terhitung dua kali.

4.6 HAVING: Menyaring Kelompok

`WHERE` menyaring **baris sebelum** dikelompokkan. `HAVING` menyaring **kelompok sesudah** dihitung. Jika syaratnya melibatkan fungsi agregat (misalnya "total belanja di atas 500 ribu"), syarat itu harus ditulis di HAVING.

Pembeli dengan total belanja di atas Rp500.000

```
SELECT pb.nama_pembeli,
       COUNT(DISTINCT ps.id_pesanan)      AS jml_pesanan,
       SUM(d.jumlah * d.harga_saat_beli)  AS total_belanja
FROM pembeli pb
JOIN pesanan ps       ON ps.id_pembeli = pb.id_pembeli
JOIN detail_pesanan d ON d.id_pesanan  = ps.id_pesanan
WHERE ps.status <> 'Dibatalkan'
GROUP BY pb.nama_pembeli
HAVING SUM(d.jumlah * d.harga_saat_beli) > 500000
ORDER BY total_belanja DESC;
```

▸ **Hasil** 3 baris · *calon member "Sultan"*

	nama_ pembeli	jml_ pesanan	total_ belanja
1	Nadia Putri	3	1687000.00
2	Maya Sari	2	825000.00
3	Rizky Ramadhan	1	569000.00

Jika syarat agregat ditulis di WHERE

```
SELECT id_pesanan, SUM(jumlah * harga_saat_beli) AS total
FROM detail_pesanan
WHERE SUM(jumlah * harga_saat_beli) > 500000
GROUP BY id_pesanan;
```

▸ **Pesan error dari PostgreSQL**

```
ERROR:  aggregate functions are not allowed in WHERE
LINE 3: WHERE SUM(jumlah * harga_saat_beli) > 500000
              ^
```

	WHERE	HAVING
Menyaring apa	Baris satu per satu	Kelompok hasil GROUP BY
Kapan dikerjakan	Sebelum GROUP BY	Sesudah GROUP BY
Boleh memakai fungsi agregat?	Tidak	Ya
Contoh	`WHERE status <> 'Dibatalkan'`	`HAVING SUM(...) > 500000`

4.7 Urutan Kerja Sebuah Query

Mengapa agregat tidak boleh di WHERE? Mengapa alias dari SELECT tidak bisa dipakai di WHERE? Jawabannya ada pada urutan PostgreSQL mengerjakan query, yang **berbeda dari urutan kita menulisnya**.



Gambar 4.3 Urutan logis pengerjaan query

WHERE (langkah 2) dikerjakan sebelum GROUP BY (langkah 3), sehingga saat WHERE berjalan belum ada kelompok yang bisa dijumlahkan. SELECT (langkah 5) dikerjakan setelah WHERE, sehingga alias yang dibuat di SELECT belum dikenal oleh WHERE:

Alias dipakai di WHERE

```
SELECT nama_produk, harga_katalog * stok AS nilai_stok
FROM produk
WHERE nilai_stok > 5000000;
```

▸ **Pesan error dari PostgreSQL**

```
ERROR:  column "nilai_stok" does not exist
LINE 3: WHERE nilai_stok > 5000000;
              ^
```

Sebaliknya, ORDER BY (langkah 6) dikerjakan setelah SELECT, sehingga **alias boleh dipakai di ORDER BY**, seperti `ORDER BY omzet DESC` di contoh 4.5. Untuk memperbaiki query di atas, tulis ulang ekspresinya: `WHERE harga_katalog * stok > 5000000`.

4.8 Latihan Blok 4 (10 Menit)

LATIHAN | Tulis query untuk setiap pertanyaan

1. Nama produk, nama toko, dan harga untuk semua produk dari toko yang berada di **Jakarta** dengan harga **di atas Rp100.000**.
2. Jumlah pesanan untuk setiap **kurir**, urutkan dari yang terbanyak.
3. Total **unit** terjual untuk setiap produk (abaikan pesanan yang dibatalkan). Tampilkan hanya produk yang terjual **minimal 3 unit**.
4. **Semua** toko beserta jumlah produknya, termasuk toko yang belum punya produk (harus tampil 0, bukan 1).
5. Total belanja per **kota pembeli** untuk pesanan berstatus **Selesai**, urutkan dari yang terbesar.

PEMBAHASAN

Soal 1

```
SELECT p.nama_produk, t.nama_toko, p.harga_katalog
FROM produk p JOIN toko t ON t.id_toko = p.id_toko
WHERE t.kota_toko = 'Jakarta' AND p.harga_katalog > 100000;
```

▸ **Hasil** 3 baris

	nama_ produk	nama_ toko	harga_ katalog
1	Earbuds TWS X1	Gadget Murah ID	229000.00
2	Powerbank 10000mAh	Gadget Murah ID	159000.00
3	Smartwatch Fit S2	Gadget Murah ID	499000.00

Soal 2

```
SELECT kurir, COUNT(*) AS jml_pesanan
FROM pesanan
GROUP BY kurir
ORDER BY jml_pesanan DESC;
```

▸ **Hasil** 4 baris · *J&T dan AnterAja sama-sama 2; urutan keduanya boleh berbeda*

	kurir	jml_ pesanan
1	JNE	4
2	SiCepat	3
3	AnterAja	2
4	J&T	2

Soal 3

```
SELECT pr.nama_produk, SUM(d.jumlah) AS unit_terjual
FROM detail_pesanan d
JOIN produk pr  ON pr.id_produk  = d.id_produk
JOIN pesanan ps ON ps.id_pesanan = d.id_pesanan
WHERE ps.status <> 'Dibatalkan'
GROUP BY pr.nama_produk
HAVING SUM(d.jumlah) >= 3
ORDER BY unit_terjual DESC, pr.nama_produk;
```

▸ **Hasil** 6 baris

	nama_ produk	unit_ terjual
1	Keripik Tempe Pedas	8
2	Kopi Gayo Arabika 250g	6
3	Sambal Roa Botol	6
4	Kabel USB-C 1m	5
5	Kemeja Batik Pria	3
6	Powerbank 10000mAh	3

Soal 4

```
SELECT t.nama_toko, COUNT(p.id_produk) AS jml_produk
FROM toko t
LEFT JOIN produk p ON p.id_toko = t.id_toko
GROUP BY t.nama_toko
ORDER BY jml_produk DESC, t.nama_toko;
```

▸ **Hasil** 5 baris

	nama_ toko	jml_ produk
1	Gadget Murah ID	5
2	Batik Pekalongan Jaya	3
3	Dapur Bunda	2
4	Kopi Gayo Aceh	2
5	Rajut Cantik	0

Jebakan soal 4: jika ditulis `COUNT(*)`, Rajut Cantik akan bernilai **1**, karena LEFT JOIN tetap menghasilkan satu baris (dengan kolom produk NULL) untuk toko itu. `COUNT(p.id_produk)` hanya menghitung yang bukan NULL, sehingga hasilnya 0.

Soal 5

```
SELECT pb.kota, SUM(d.jumlah * d.harga_saat_beli) AS total_belanja
FROM pembeli pb
JOIN pesanan ps       ON ps.id_pembeli = pb.id_pembeli
JOIN detail_pesanan d ON d.id_pesanan  = ps.id_pesanan
WHERE ps.status = 'Selesai'
GROUP BY pb.kota
ORDER BY total_belanja DESC;
```

▸ **Hasil** 5 baris

	kota	total_ belanja
1	Bandung	1412000.00
2	Jakarta	569000.00
3	Surabaya	475000.00
4	Bogor	315000.00
5	Semarang	159000.00

BLOK 5 | 20 MENIT | TERPANDU

UNION, Subquery, EXISTS, ANY/ALL, dan CASE

Menggabungkan hasil, bertanya di dalam pertanyaan, dan membuat kolom bersyarat

5.1 UNION dan UNION ALL

JOIN menyambung tabel **ke samping** (menambah kolom). `UNION` menumpuk dua hasil query **ke bawah** (menambah baris). Syaratnya, kedua query harus memiliki jumlah kolom yang sama dengan tipe yang cocok. `UNION` membuang baris kembar, sedangkan `UNION ALL` mempertahankan semuanya.

Semua kota yang terlibat di Pasar Nusantara

```
SELECT kota FROM pembeli
UNION
SELECT kota_toko FROM toko
ORDER BY kota;
```

▸ **Hasil** 9 baris · *8 kota pembeli + 5 kota toko, tetapi yang kembar dibuang*

	kota
1	Bandung
2	Bogor
3	Jakarta
4	Medan
5	Pekalongan
6	Semarang
7	Surabaya
8	Takengon
9	Yogyakarta

UNION ALL dengan kolom penanda

```
SELECT kota AS nama_kota, 'Pembeli' AS peran FROM pembeli
UNION ALL
SELECT kota_toko, 'Toko' FROM toko
ORDER BY nama_kota, peran;
```

▸ **Hasil** 13 baris (ditampilkan 7 baris pertama)

	nama_ kota	peran
1	Bandung	Pembeli
2	Bandung	Toko
3	Bogor	Pembeli
4	Bogor	Pembeli
5	Bogor	Toko
6	Jakarta	Pembeli
7	Jakarta	Toko

5.2 Subquery: Query di Dalam Query

Subquery adalah SELECT yang ditulis di dalam tanda kurung, lalu hasilnya dipakai oleh query luar. Contoh klasiknya: "produk yang harganya di atas rata-rata". Rata-ratanya harus dihitung dulu, baru dibandingkan.

Subquery yang menghasilkan satu nilai

```
SELECT nama_produk, harga_katalog
FROM produk
WHERE harga_katalog > (SELECT AVG(harga_katalog) FROM produk)
ORDER BY harga_katalog DESC;
```

▸ **Hasil** 5 baris · *rata-rata = 158.416,67*

	nama_ produk	harga_ katalog
1	Smartwatch Fit S2	499000.00
2	Kain Batik Tulis	350000.00
3	Earbuds TWS X1	229000.00
4	Kemeja Batik Pria	185000.00
5	Powerbank 10000mAh	159000.00

Subquery juga bisa menghasilkan **satu kolom berisi banyak nilai**, lalu dipakai bersama `IN`:

Pembeli yang pernah memakai JNE

```
SELECT nama_pembeli
FROM pembeli
WHERE id_pembeli IN (SELECT id_pembeli FROM pesanan WHERE kurir = 'JNE')
ORDER BY nama_pembeli;
```

▸ **Hasil** 4 baris

	nama_ pembeli
1	Dimas Pratama
2	Maya Sari
3	Nadia Putri
4	Rizky Ramadhan

5.3 EXISTS dan NOT EXISTS

`EXISTS (subquery)` bernilai benar jika subquery menghasilkan **minimal satu baris**. Isi kolomnya tidak penting, sehingga orang biasa menulis `SELECT 1`. Subquery EXISTS biasanya **merujuk ke tabel luar** (perhatikan `ps.id_pembeli = pb.id_pembeli`), jadi ia dijalankan untuk setiap baris luar. Bacalah seperti kalimat biasa: "tampilkan pembeli **yang ada** pesanannya berisi produk Elektronik".

Pembeli yang pernah membeli Elektronik

```
SELECT pb.nama_pembeli
FROM pembeli pb
WHERE EXISTS (
  SELECT 1
  FROM pesanan ps
  JOIN detail_pesanan d ON d.id_pesanan = ps.id_pesanan
  JOIN produk p         ON p.id_produk  = d.id_produk
  WHERE ps.id_pembeli = pb.id_pembeli
    AND p.kategori = 'Elektronik'
)
ORDER BY pb.nama_pembeli;
```

▸ **Hasil** 3 baris

	nama_ pembeli
1	Fajar Nugroho
2	Nadia Putri
3	Rizky Ramadhan

NOT EXISTS: toko yang belum punya produk

```
SELECT t.nama_toko
FROM toko t
WHERE NOT EXISTS (SELECT 1 FROM produk p WHERE p.id_toko = t.id_toko);
```

▸ **Hasil** 1 baris · *hasilnya sama dengan RIGHT JOIN di 4.3*

	nama_ toko
1	Rajut Cantik

TIPS | LEFT JOIN ... IS NULL atau NOT EXISTS?

Keduanya menjawab pertanyaan "yang belum pernah…". NOT EXISTS biasanya lebih mudah dibaca, dan aman dari jebakan NULL yang bisa muncul pada `NOT IN`. Pilih yang paling jelas bagi kalian. Hasilnya harus sama.

5.4 ANY dan ALL

`ANY` dan `ALL` membandingkan satu nilai dengan **sekumpulan** nilai dari subquery. `> ALL (...)` berarti lebih besar dari **semua** nilai, atau dengan kata lain lebih besar dari yang terbesar. `> ANY (...)` berarti lebih besar dari **minimal satu** nilai, atau lebih besar dari yang terkecil.

> ALL: lebih mahal dari SEMUA produk Fashion

```
SELECT nama_produk, harga_katalog
FROM produk
WHERE harga_katalog > ALL (SELECT harga_katalog FROM produk WHERE kategori = 'Fashion');
```

▸ **Hasil** 1 baris · *produk Fashion termahal = 350.000*

	nama_produk	harga_katalog
1	Smartwatch Fit S2	499000.00

> ANY: lebih mahal dari SETIDAKNYA SATU produk Elektronik

```
SELECT nama_produk, kategori, harga_katalog
FROM produk
WHERE kategori <> 'Elektronik'
  AND harga_katalog > ANY (SELECT harga_katalog FROM produk WHERE kategori = 'Elektronik')
ORDER BY harga_katalog;
```

▸ **Hasil** 6 baris · *Elektronik termurah = 35.000, sehingga Keripik Tempe (20.000) tidak lolos*

	nama_produk	kategori	harga_katalog
1	Sambal Roa Botol	Makanan	45000.00
2	Kopi Gayo Robusta 250g	Makanan	75000.00
3	Kopi Gayo Arabika 250g	Makanan	95000.00
4	Daster Batik Cap	Fashion	120000.00
5	Kemeja Batik Pria	Fashion	185000.00
6	Kain Batik Tulis	Fashion	350000.00

5.5 CASE: Kolom Bersyarat

`CASE` adalah "if-else"-nya SQL. Kondisi diperiksa dari atas ke bawah, dan kondisi **pertama** yang benar menentukan hasilnya. `ELSE` menangkap sisanya.

Label kelas harga

```
SELECT nama_produk, harga_katalog,
       CASE
         WHEN harga_katalog < 50000  THEN 'Murah'
         WHEN harga_katalog < 200000 THEN 'Sedang'
         ELSE 'Premium'
       END AS kelas_harga
FROM produk
ORDER BY harga_katalog;
```

▸ **Hasil** 12 baris

	nama_produk	harga_katalog	kelas_harga
1	Keripik Tempe Pedas	20000.00	Murah
2	Kabel USB-C 1m	35000.00	Murah
3	Sambal Roa Botol	45000.00	Murah
4	Kopi Gayo Robusta 250g	75000.00	Sedang
5	Mouse Wireless M2	89000.00	Sedang
6	Kopi Gayo Arabika 250g	95000.00	Sedang
7	Daster Batik Cap	120000.00	Sedang
8	Powerbank 10000mAh	159000.00	Sedang
9	Kemeja Batik Pria	185000.00	Sedang
10	Earbuds TWS X1	229000.00	Premium
11	Kain Batik Tulis	350000.00	Premium
12	Smartwatch Fit S2	499000.00	Premium

Peringatan stok untuk penjual

```
SELECT nama_produk, stok,
       CASE WHEN stok = 0  THEN 'Habis'
            WHEN stok < 20 THEN 'Menipis'
            ELSE 'Aman' END AS status_stok
FROM produk
WHERE stok < 40
ORDER BY stok;
```

▸ **Hasil** 6 baris

	nama_ produk	stok	status_ stok
1	Kopi Gayo Robusta 250g	0	Habis
2	Smartwatch Fit S2	8	Menipis
3	Kain Batik Tulis	12	Menipis
4	Earbuds TWS X1	25	Aman
5	Daster Batik Cap	30	Aman
6	Sambal Roa Botol	35	Aman

CASE juga bisa dipakai **di dalam** fungsi agregat untuk menghitung beberapa kategori sekaligus dalam satu baris. Ini teknik laporan yang sangat berguna:

Rekap status pesanan dalam satu baris

```
SELECT COUNT(*)                                                  AS semua,
       SUM(CASE WHEN status = 'Selesai' THEN 1 ELSE 0 END)       AS selesai,
       SUM(CASE WHEN status IN ('Diproses', 'Dikirim') THEN 1 ELSE 0 END) AS berjalan,
       SUM(CASE WHEN status = 'Dibatalkan' THEN 1 ELSE 0 END)    AS batal
FROM pesanan;
```

▸ **Hasil** 1 baris

	semua	selesai	berjalan	batal
1	11	6	4	1

5.6 Bonus: Fungsi Kecil yang Sering Dibutuhkan

Tanggal: ambil bagian dan hitung selisih hari

```
SELECT id_pesanan, tgl_pesanan,
       EXTRACT(MONTH FROM tgl_pesanan)   AS bulan,
       DATE '2026-06-30' - tgl_pesanan   AS umur_hari
FROM pesanan
WHERE status IN ('Diproses', 'Dikirim');
```

▸ **Hasil** 4 baris · *selisih dua DATE menghasilkan jumlah hari*

	id_ pesanan	tgl_ pesanan	bulan	umur_ hari
1	ORD-008	2026-06-14	6	16
2	ORD-009	2026-06-18	6	12
3	ORD-010	2026-06-20	6	10
4	ORD-011	2026-06-21	6	9

Jebakan pembagian bilangan bulat

```
SELECT 7 / 2 AS bagi_bulat, 7 / 2.0 AS bagi_desimal, ROUND(7 / 2.0, 1) AS dibulatkan;
```

▸ **Hasil** 1 baris

	bagi_ bulat	bagi_ desimal	dibulatkan
1	3	3.5000000000000000	3.5

Pembagian dua **bilangan bulat** di PostgreSQL menghasilkan bilangan bulat, dan bagian desimalnya **dibuang**. `7 / 2` menghasilkan 3, bukan 3,5. Ini sering membuat persentase bernilai 0. Solusinya, jadikan salah satu operan desimal (`7 / 2.0`) atau ubah tipenya dengan `::numeric`.

5.7 Latihan Blok 5 (5 Menit)

LATIHAN | Tulis query untuk setiap pertanyaan

1. Dengan **NOT EXISTS**, tampilkan produk yang belum pernah dipesan siapa pun.
2. Tampilkan setiap pesanan beserta totalnya, dan beri label **"Besar"** jika total ≥ Rp500.000, selain itu **"Kecil"**.
3. Buat satu daftar berisi nama **semua pembeli dan semua toko** dalam kolom `nama_pihak`, dengan kolom `peran` yang berisi "Pembeli" atau "Toko". Urutkan berdasarkan nama.

PEMBAHASAN

Soal 1

```
SELECT p.id_produk, p.nama_produk
FROM produk p
WHERE NOT EXISTS (SELECT 1 FROM detail_pesanan d WHERE d.id_produk = p.id_produk);
```

▸ **Hasil** 1 baris

	id_ produk	nama_ produk
1	PR12	Mouse Wireless M2

Soal 2

```
SELECT id_pesanan,
       SUM(jumlah * harga_saat_beli) AS total,
       CASE WHEN SUM(jumlah * harga_saat_beli) >= 500000 THEN 'Besar' ELSE 'Kecil' END AS
label
FROM detail_pesanan
GROUP BY id_pesanan
ORDER BY id_pesanan;
```

▸ **Hasil** 11 baris (ditampilkan 6 baris pertama)

	id_ pesanan	total	label
1	ORD-001	579000.00	Besar
2	ORD-002	159000.00	Kecil
3	ORD-003	833000.00	Besar
4	ORD-004	569000.00	Besar
5	ORD-005	475000.00	Kecil
6	ORD-006	229000.00	Kecil

Soal 3

```
SELECT nama_pembeli AS nama_pihak, 'Pembeli' AS peran FROM pembeli
UNION ALL
SELECT nama_toko, 'Toko' FROM toko
ORDER BY nama_pihak;
```

▸ **Hasil** 13 baris (ditampilkan 6 baris pertama)

	nama_ pihak	peran
1	Anisa Rahmawati	Pembeli
2	Batik Pekalongan Jaya	Toko
3	Dapur Bunda	Toko
4	Dimas Pratama	Pembeli
5	Fajar Nugroho	Pembeli
6	Gadget Murah ID	Toko

Rangkuman: Anatomi Lengkap SELECT

Kerangka SELECT (angka = urutan pengerjaan)

```
SELECT   [DISTINCT] kolom, ekspresi AS alias, AGREGAT(...)   -- 5. pilih & hitung
FROM     tabel_a a                                          -- 1. sumber data
[INNER | LEFT | RIGHT | FULL] JOIN tabel_b b ON b.fk = a.pk
WHERE    kondisi baris                                      -- 2. saring baris
GROUP BY kolom                                              -- 3. kelompokkan
HAVING   kondisi kelompok (boleh agregat)                   -- 4. saring kelompok
ORDER BY kolom [ASC | DESC]                                 -- 6. urutkan
LIMIT    n OFFSET m;                                        -- 7. potong
```

KESALAHAN UMUM

1. `= NULL` **alih-alih** `IS NULL`**.** Query tidak error, tetapi hasilnya selalu kosong. Ini jenis kesalahan yang paling berbahaya karena diam-diam salah.
2. **Lupa tanda kurung saat mencampur AND dan OR**, sehingga baris yang tidak diinginkan ikut lolos.
3. **Kolom di SELECT yang tidak ada di GROUP BY dan tidak diagregasi.** Baca pesan error-nya: PostgreSQL menyebut nama kolomnya.
4. `COUNT(*)` **setelah LEFT JOIN**, sehingga yang tidak punya pasangan terhitung 1. Gunakan `COUNT(kolom_tabel_kanan)`.
5. **JOIN tanpa ON yang benar** menghasilkan baris berlipat. Selalu periksa: apakah jumlah baris hasil masuk akal?
6. **Menghitung dengan harga katalog, bukan harga saat beli**, untuk omzet atau total belanja. Ingat pelajaran snapshot di Pertemuan 5.